Universidad Nacional Autónoma de México.
Facultad de Ciencias.

Computación distribuida.

Profesor: Fernando Michel Tavera.
Ayudante de laboratorio: Luis Mario Escobar Rosales .



Práctica 02

Algoritmos básicos y ordenamientos.
Fecha: 20 de octubre del 2026.

Alumnos:

Jesús Echeverría Verduzco	**No. Cuenta:** 321239167
José Carlos Pinzón Chan	**No. Cuenta:** 425122448

Durante esta práctica, se terminó la implementación de algoritmos básicos de búsqueda y ordenamiento distribuidos sobre una topología de estrella. Además, también se implementaron las funciones faltantes para un algoritmo ConvergeCast sencillo, que funciona en una topología de árbol.

Igualmente, se implementaron 2 algoritmos secuenciales, uno de búsqueda y el otro de ordenamiento, que son utilizados por los nodos trabajadores para realizar las búsquedas y ordenamientos sobre los segmentos que reciben.

Nota: En algunas de las descripciones, el orden real en el que suceden las cosas en el programa puede variar. También puede ocurrir que se omitan algunos detalles de lo que en realidad hace el programa, en tal situación, puede consultar el código y su documentación.

Detalles:

- **Búsqueda distribuida:**

  Hacía falta implementar las funciones **`nodo_busqueda_iniciar`** y **`procesar_trabajo_busqueda`**.

  - **`nodo_busqueda_iniciar`** : Esta función inicia la propagación de la búsqueda de un elemento en un arreglo desde el nodo coordinador. Si el nodo desde el que se quiere iniciar no es el coordinador no hace nada pero regresa true.

    Esta función crea una cuadrícula que almacena los segmentos del arreglo que se pretenden enviar a los nodos trabajadores. Después envía dichos segmentos a su respectivo nodo. Una vez finalizados los envíos, elimina la cuadrícula y regresa **`true`** indicando que la búsqueda fue iniciada correctamente.

  - **`procesar_trabajo_busqueda`** : Función que simula el comportamiento de un nodo al recibir un mensaje de búsqueda.

El nodo realiza la búsqueda del elemento proporcionado por el mensaje en el segmento también, proporcionado por el mensaje . Se crea entonces un mensaje dirigido al nodo coordinador, en dicho mensaje se almacena el resultado de la búsqueda. Finalmente se envía el mensaje al coordinador.

- **Ordenamiento distribuido:**

  Aquí solamente se implementó la función **`nodo_ordenamiento_iniciar`** .

  - **`nodo_ordenamiento_iniciar`** : Función que inicia la propagación del ordenamiento de un arreglo desde el nodo coordinador. Análogamente a la función **`nodo_busqueda_iniciar`**, esta función regresa **`true`** si el nodo dado no es el coordinador.

    Se crea una cuadrícula para segmentar el arreglo. Se envían a los nodos trabajadores (vecinos del nodo coordinador) sus segmentos correspondientes en un mensaje de ordenamiento. Si todos los mensajes pudieron ser enviados, entonces procedemos a eliminar la cuadrícula y regresar **`true`**, ya que el ordenamiento fue iniciado correctamente.

- **ConvergeCast:**

  Para ConvergeCast fue necesario implementar dos funciones:

  - **`nodo_convergecast_iniciar`** : Parecida a sus homólogas de ordenamiento y búsqueda.

    Verificamos que el nodo no haya terminado de propagar o bien de esperar respuestas. Si este es el caso, toca ver si es el nodo raíz, una hoja o un nodo interno. Si es la raíz o un nodo interno no hacemos nada y regresamos **`true`**.

    En caso de ser una hoja, debemos iniciar con la propagación. Como excepción tenemos el caso en el que el nodo es una hoja y la raíz. En ese caso simplemente regresamos **`true`**.

    Para propagar el acumulador, las hojas deben enviarlo a su padre, este último a su padre y así hasta que lleguemos a la raíz. Como de momento sólo se desea iniciar este proceso, las hojas realizan un único envío a su padre y marcan a su estado como **`finalizo`**. Después se regresa **`true`** para señalar el inicio exitoso del ConvergeCast en esa hoja.

  - **`recibir_convergecast`** : Parecida a su homóloga de búsqueda.

    Cuando un nodo recibe un mensaje del tipo ConvergeCast, se verifica que el nodo que lo haya recibido siga esperando por respuestas de sus nodos hijos, osea que no haya finalizado. Sea este el caso,entonces existe al menos un hijo cuyo mensaje sigue esperando. Vemos si el mensaje recibido proviene

de algún hijo del cual todavía no hemos recibido mensaje. En tal caso, combinamos el acumulado del mensaje con el acumulado del nodo.

En algún punto el padre ya ha recibido todos los mensajes de sus hijos, es necesario entonces propagar el ConvergeCast al padre del nodo. Sí el nodo es la raíz, eureka, hemos terminado. Si no es así, enviamos el acumulado al padre y se marca como `finalizo` el estado del nodo.

- **Funciones auxiliares:**

  Las funciones que utiliza cada nodo para ordenar y buscar respectivamente, son:

  - **`ordenar_inserccion`** : Función que ordena un arreglo cualquiera de enteros. Para realizar este ordenamiento se utilizó el algoritmo SelectionSort. Asimismo fue necesario definir las siguientes funciones auxiliares:
    - ★ **`busca_minimo`** : Regresa la posición del elemento más pequeño entre las posiciones **a** y **b** de un arreglo.
    - ★ **`intercambiar`** : Intercambia a los elementos en las posiciones **r** y **k** de un arreglo. No se verifica que r y k se encuentren dentro de los límites del arreglo ya que esta función es utilizada por **`ordenar_inserccion`** que ya asegura esta cualidad.

    Nota: Se implementó búsqueda lineal también aunque no se usa.

  - **`busqueda_binaria`** : Implementación iterativa de la búsqueda binaria sobre un arreglo. Lamentablemente, para utilizar búsqueda binaria, es imprescindible que el arreglo ya esté ordenado por lo que primero debemos ordenar el arreglo utilizando **`ordenar_inserccion`** lo que repercute en su complejidad.

    Por si esto fuera poco, hacemos una copia del arreglo a buscar para no modificar el original. Esto lo hicimos con la idea de evitar inconvenientes después en las implementaciones de los algoritmos distribuidos, no obstante, parece ser que podría omitirse este paso y trabajar directamente con el arreglo.